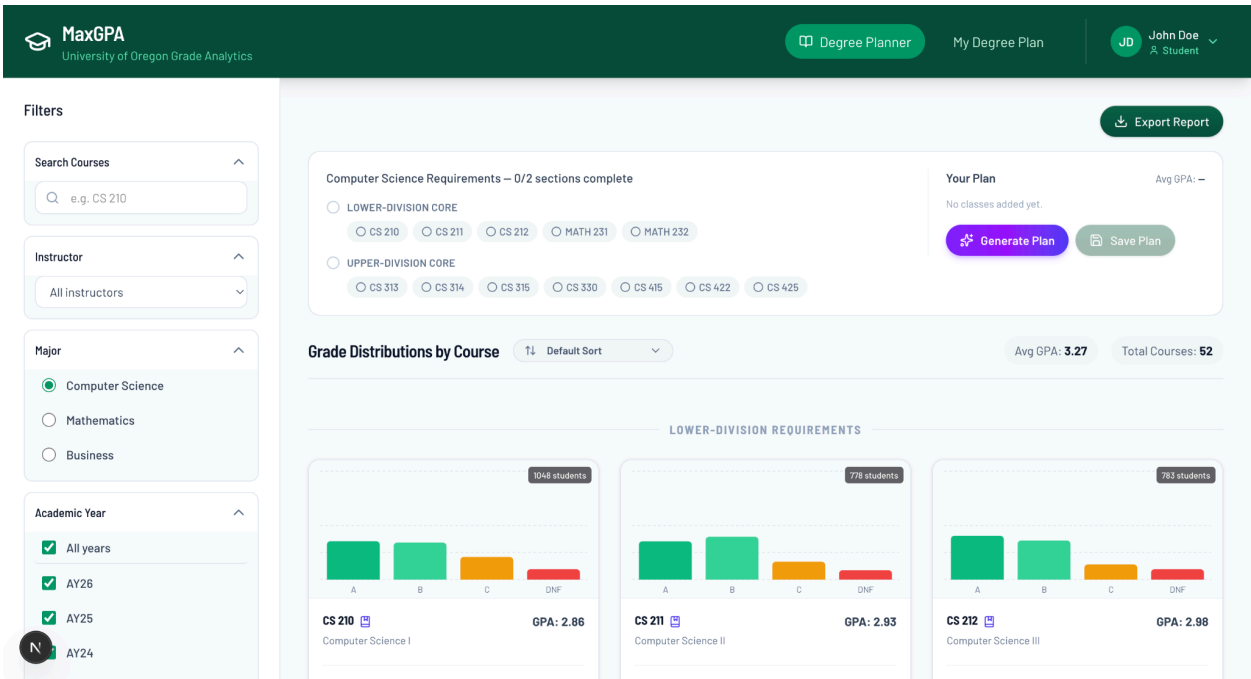


Group 7 Project Plan



1

Authors:

Kaegan Koski, Robert Mimms, Hiro Nakae, Peyton Phillips

Date:

05/04/2026

¹ Finished product image

Table of Contents

Table of Contents	1
Revisions History	2
Management	2
Work Schedule	4
Monitoring + Reporting	5
Build Plan	5
Risk and Risk Mitigation:	5
Special notes:	6
Notes 4/13	6
Notes 4/14	6
Notes 4/26	6
Notes 4/28	7
Notes 5/1	7
Notes 5/4	7

Revisions History

Date	Author	Description
4/15/2026	Group 7	Initial submission
4/29/2026	Group 7	Final submission created
5/05/2026	Group 7	Reformatted to 2 documents

Management

I. People

A. Peyton Phillips

1. FastAPI

- a) FastAPI is used for getting stat files from backend for use with other libraries
- b) GET `/api/major/course/{course_code}`: returns the average grade for that course
- c) GET `/api/major/instructor/{instructor_id}`: returns average grades given by that teacher.

2. Creates visualization for data using PANDAS and NumPy libraries

- a) PANDAS is used for parsing CSV files that are gotten from backend, as well as putting the data into graph format
- b) NumPy is used for any math functions that could be needed (linear algebra, vector multiplication, etc.)

B. Robert Mimms

- 1. SQLite3 Database + Queryable data
- 2. Turn the CSV into a schema design
- 3. Write script to parse CSV
- 4. Seed database

C. Kaegan Koski

1. Testing of code

- a) Making sure all parts work together
- b) Integrating parts together if they don't

2. Documentation

- a) Takes notes, writes down general goals to keep (plans)

- b) Actively changes plan if needed, otherwise just monitors

D. Hiro Nakae

1. Design

- a) Student and Admin Pages.
- b) Fetching and displaying data on the graphs.

2. Implementation

- a) Web(client) pages, client-side api calls, chart library integration.
- b) Use a Next.js Server Component to fetch() that data.
- c) Pass that JSON to a Client Component where Chart.js renders the bar graph.

3. Testing

- a) Use end-to-end testing to verify dynamic ui displays correct data.

4. Documentation

II. Structure

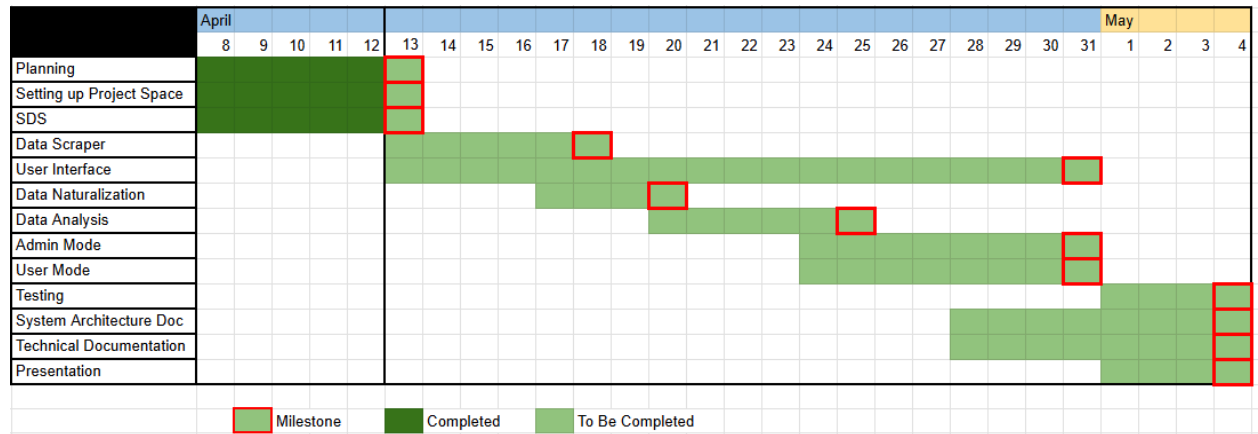
A. Roles

1. At time of writing, a 'manager' role doesn't seem needed. People are willing to fill in the role as needed later on though.
2. Mainly, people will work in their specifications described above.
However, if a part of the project falls behind the Gantt chart, others can step in to work with the person in that area until caught up

B. Team based

1. We meet in-person weekly at minimum, most likely 2 meeting per week
2. Communication with each other (when not in person) is done over text
3. Meetings are 1 hour long
4. Mandatory reports are done by Peyton

Work Schedule



2

- Key

- P → Peyton
- K → Kaegan
- R → Robert
- H → Hiro
- G → Group / All

- Milestones

- Planning (G)
- Setting up Project Space (G)
- SDS (G)
- Data Scraper (R)
- User Interface (H)
- Data Naturalization (P)
- Data Analysis (P)
- Admin Mode (H)
- User Mode (H)
- Testing (K)
- System Architecture Documentation (K)
- Technical Documentation (K)
- Presentation (G)

² Gantt Chart of the schedule

Monitoring + Reporting

- Github commits will be reviewed weekly to track code contributions
- Task_And_Assignment_Breakdown will track:
 - Current status on assigned work
 - Any changes to expected timeline outside of the Gantt chart
- Weekly meeting notes will include:
 - Attendees
 - Decisions made
 - Completed tasks
 - Blockers

Build Plan

The planning phase is a significant portion of the schedule so as to highlight the importance of this step. The first project deliverables can all be worked on in parallel, so they are indicated as such.

As soon as the planning phase is completed, work can begin on the database. Once the data can be fully loaded into the system, the API layer can be implemented which includes naturalization and parts of analysis. The rest of analysis includes generation of the distribution tables which can only be made when the API layer is complete. These tasks all lay on the critical path.

Once these systems are implemented, the user and admin mode can be implemented using the database, API layer, and distribution generation. Testing can be done in parallel with most of the things on this list.

The user interface, while reliant on all of these systems, can be designed and prepared to a large extent even before each of these subsystems are complete. This is indicated by the user interface being parallel to every other milestone on the list.

Finally, the implementation of documentation does require that the system is stable and finished in design. The presentation and documentation will become the main focus of the final days of the project.

Risk and Risk Mitigation:

1. Inconsistency in the CSV data files
 - Risk: malformed or missing data could break ingestion
 - Mitigation: implement validation checks before database verification
2. API performance bottlenecks

- Risk: slow queries on large datasets
- Mitigation: optimize queries and precompute aggregates
- 3. Integration issues between frontend and backend
 - Risk: mismatched data formats
 - Mitigation: define strict API contracts early
- 4. Team coordination delays
 - Risk: dependencies blocking progress
 - Mitigation: weekly check ins + a shared task tracker

Special notes:

- We did not start doing these notes until the middle of week 2 and we did not meet 2 times during week 2 so there are no notes from week 2. A general summary of what happened is we decided roles, Kaegan's info got put into our group, made a repo, created general documentation
- Hiro made a prototype of the project with basic functionality
- No blockers

Notes 4/13

- Hiro, Robert, Peyton met from 1:30 to 2 at the knight library
- Worked on project pitch together during this time
- Kaegan couldn't make it today (busiest on Mondays) but contributed with the Gantt chart
- No blockers

Notes 4/14

- Project proposal went well, we got ideas for more team communication / ticketing
- Everyone was in attendance (Kaegan, Robert, Peyton, Hiro)
- Initial submission was formatted by Peyton, waiting for teammates to input some info before submitting
- No blockers

Notes 4/26

- Talked about remaining fixes to make
- Everyone was in attendance (Kaegan, Robert, Peyton, Hiro)
- No blockers

Notes 4/28

- Meeting with TA / shareholders
- No blockers

Notes 5/1

- Small, in-person meeting for writing debug cases for the API at the Knight Library (Kaegan, Peyton)
- Assigned what cases needed to have tests, who would write what.
- No blockers

Notes 5/4

- Everyone in attendance (Kaegan, Robert, Peyton, Hiro)
- Talked about finishing up the final submission (Peyton in charge of Project Plan + SDS, Robert and Kaegan running the program using the instructions from the user document, Hiro wrote the user document)
- No blockers